

DEVOPS MASTER COURSE

Volume 6C

AWS Database & Data Platforms

Chapters 21-30

RDS | AURORA | DYNAMODB | CACHE | REDSHIFT | LAKES | STREAMS | MIGRATION

Production databases, analytics, migration, governance, labs, and interviews

2026 Edition

About This Part

Volume 6C advances from infrastructure into database and data-platform engineering. It covers relational systems, purpose-built NoSQL, caching, warehouses, lakes, streaming, search, migration, security, recovery, cost, and an integrated capstone. The focus is production decision-making: semantics, ownership, consistency, failure modes, observability, migration safety, and measurable recovery.

Scope and Safety

- Use a sandbox account and create budgets before databases, Redshift, OpenSearch, MSK, MemoryDB, NAT gateways, or cross-Region resources.
- Verify current engine versions, regional availability, quotas, and pricing in official AWS documentation.
- Use private connectivity, TLS, managed secrets, encryption, and least privilege.
- Never use production data in labs unless it is approved, masked, and governed.
- Export evidence and delete chargeable resources after validation.

How to Use the Labs

- Write expected access, consistency, SLO, RPO, RTO, security, and cost before provisioning.
- Tag every resource with owner, environment, application, cost center, and expiry.
- Measure behavior during failure, not only successful creation.
- Validate data correctness and business transactions, not merely service status.
- Repeat important deployments using Infrastructure as Code.

Table of Contents

Chapter 21 Amazon RDS Engineering

Chapter 22 Amazon Aurora Engineering

Chapter 23 Amazon DynamoDB and NoSQL Design

Chapter 24 Amazon ElastiCache and MemoryDB

Chapter 25 Amazon Redshift and Data Warehousing

Chapter 26 AWS Data Lakes and Analytics

Chapter 27 Streaming, Kafka, and Search Analytics

Chapter 28 AWS Database Migration Engineering

Chapter 29 Data Security, Backup, Reliability, and Operations

Chapter 30 Enterprise Data Platform Architecture and Capstone

Appendices CLI Reference, Troubleshooting Matrix, Readiness Checklist, Glossary, and Sources

Navigation: Heading styles allow Word or LibreOffice to generate a field-based table of contents.

Amazon RDS Engineering

Managed relational databases, availability, scaling, connectivity, backups, and production operations

VOLUME 6C | CHAPTER 21

Learning Objectives

- Explain what Amazon RDS manages and what remains the customer responsibility.
- Select engines, instance classes, storage, Multi-AZ models, and read replicas.
- Design private connectivity, credentials, encryption, and connection management.
- Operate backups, maintenance, upgrades, monitoring, and failure response.

21.1 Managed relational database responsibility

Amazon Relational Database Service runs supported relational database engines while automating infrastructure tasks such as provisioning, replacement, patch workflows, backup orchestration, and monitoring integration. It does not remove the need for schema design, query tuning, transaction discipline, index management, data classification, access control, capacity planning, and recovery testing.

The right mental model is a managed database platform, not a database that operates itself. Engineers still own application behavior, workload isolation, connection storms, slow queries, lock contention, engine compatibility, and release safety. Production teams therefore define service standards around engine versions, parameter groups, option groups where applicable, networking, encryption, observability, backup retention, and maintenance windows.

21.2 Engine, compute, and storage decisions

RDS supports several relational engines, each with different features, licensing, extension ecosystems, replication behavior, and operational constraints. Select from application requirements and migration feasibility rather than personal familiarity. The database engine choice affects SQL compatibility, transaction semantics, stored code, tooling, skills, and exit options.

DB instance class selection combines CPU, memory, network, and storage bandwidth. Burstable classes may fit low-duty development systems but can hide sustained-capacity problems. Storage type, allocated size, IOPS, throughput, autoscaling limits, and engine-specific behavior must be tested with realistic query and transaction patterns. A database can be CPU-idle while storage latency or lock contention dominates user response time.

21.3 High availability and read scaling

A Multi-AZ DB instance deployment maintains a synchronous standby in another Availability Zone and uses a single endpoint that changes destination during failover. The standby is for high availability and normally does not serve read traffic. A Multi-AZ DB cluster uses a writer and readable replicas across three Availability Zones according to supported engine and configuration options.

Read replicas use asynchronous replication and are primarily a read-scaling and migration mechanism. They can also contribute to disaster-recovery designs, but replica lag and unreplicated failures must be considered. A read replica is not equivalent to a synchronous standby, and promotion is an operational event that changes recovery and application behavior.

21.4 Connectivity, credentials, and pooling

Place production databases in private subnets with a DB subnet group spanning the Availability Zones required by the design. Permit inbound traffic only from approved application security groups or network paths. Public accessibility should be an explicit exception, not a convenience default.

Applications should retrieve credentials from a managed secret system and rotate them through a tested workflow. IAM database authentication is useful for supported engines and access patterns but does not remove database authorization. RDS Proxy can pool and reuse connections for supported engines, reduce connection churn, and improve resilience during failover; it must be configured with correct secrets, network access, TLS, and application behavior.

21.5 Backup, maintenance, and observability

Automated backups provide point-in-time recovery within the configured retention window. Manual snapshots persist until explicitly removed. Snapshot copy and export patterns support account, Region, analytics, and governance requirements, but KMS permissions and service support must be validated. A successful backup job is not proof that the application can be recovered.

CloudWatch metrics, database logs, Enhanced Monitoring, Performance Insights or supported performance tooling, event notifications, and engine-native views should be combined. Maintenance windows and engine upgrades require compatibility testing, rollback planning, parameter review, and business communication. Minor and major upgrades are application changes, even when the platform performs much of the infrastructure work.

RDS deployment choices

Pattern	Primary objective	Key caution
Single-AZ DB instance	Development, noncritical, or externally replicated workload	Instance/AZ interruption causes longer outage
Multi-AZ DB instance	Synchronous standby and managed failover	Standby is not general read capacity
Multi-AZ DB cluster	HA plus readable replicas for supported engines	Different topology, pricing, and engine support
Read replica	Read scaling, reporting, migration, regional copy	Asynchronous lag and separate promotion process
RDS Proxy	Connection pooling and failover resilience	Supported engines, auth, pooling behavior, and cost

Enterprise Architecture

Production RDS service

Application Auto Scaling fleet in private subnets

V

RDS Proxy or controlled client pool with TLS

V

Multi-AZ RDS writer and standby/readers across approved AZs

V

Secrets Manager, KMS, IAM, parameter groups, and change pipeline

V

CloudWatch, database logs, performance analysis, backups, restore tests, and runbooks

RDS inspection commands

```
aws rds describe-db-instances
aws rds describe-db-clusters
aws rds describe-db-snapshots --snapshot-type manual
aws rds describe-events --duration 1440
aws rds describe-db-proxies

# Validate the active identity before any change
aws sts get-caller-identity
```

Production Best Practices

- Use private subnets and least-privilege security-group references.
- Enable encryption at creation with a governed KMS key.
- Separate HA from read-scaling requirements.
- Set backup retention and test point-in-time restore.
- Use managed secrets and TLS; never store passwords in AMIs or user data.
- Create alarms for storage, connections, latency, replication lag, failed logins, and availability signals.
- Apply schema changes with migration tooling, review, and rollback.
- Test failover with the actual application connection pool.

Troubleshooting

Symptom	Likely cause	Evidence / action
Application timeout	Route, SG, DNS, TLS, credentials, or connection exhaustion	Resolve endpoint, test port/TLS, inspect SG/flow logs and database connection count

Symptom	Likely cause	Evidence / action
Failover takes too long	Client DNS caching or pool retry behavior	Measure DNS refresh, TCP timeout, driver failover, and proxy behavior
CPU low but latency high	Storage wait, locks, query plan, or connection queue	Use performance tooling, engine wait events, slow logs, and storage metrics
Read replica stale	Replication lag or long transaction	Inspect replica lag, source workload, network, and blocked apply
Backup restore fails	KMS, snapshot state, subnet/parameter mismatch	Inspect event details, key policy, target configuration, and quotas
Storage full	Growth, temp files, logs, or autoscaling ceiling	Inspect free storage, workload growth, log retention, and max threshold

Hands-on Lab: Deploy and fail over a private Multi-AZ RDS database

Objective: Create an encrypted private database, connect through a controlled application path, observe health, test failover, and restore a backup.

Implementation Tasks

1. Create a DB subnet group spanning at least two private subnets.
2. Create a database security group that accepts only the application security group.
3. Launch an encrypted Multi-AZ test database with backup retention and log exports.
4. Store credentials in Secrets Manager and connect using TLS from a Session Manager host.
5. Create a small schema and generate controlled read/write traffic.
6. Capture metrics and trigger a supported failover test.
7. Create a snapshot or point-in-time restore into a separate validation database.

Validation

- No public endpoint or management port is exposed.
- Application reconnects after failover within the recorded objective.
- Logs and metrics show the event and recovery.
- Restored data passes row-count and application checks.

Cleanup

- Export evidence, then remove validation DB, primary DB, snapshots not retained, proxy, secrets, security groups, and temporary host.
- Confirm final snapshots and KMS keys follow the retention decision.

Interview Questions

1. What does RDS manage and what does the customer still own?
2. How does Multi-AZ differ from a read replica?

3. When would you use a Multi-AZ DB cluster?
4. Why can connection pools delay failover recovery?
5. What problem does RDS Proxy solve?
6. How do automated backups differ from manual snapshots?
7. How would you investigate database latency with low CPU?
8. Why must database upgrades be treated as application changes?

Chapter Summary

- RDS reduces infrastructure operations but does not replace database engineering.
- High availability, read scaling, backup, and disaster recovery are separate design concerns.
- Private connectivity, managed secrets, TLS, and connection control are production defaults.
- Recovery and failover must be validated with the real application.

Official AWS References

Amazon RDS User Guide - <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Welcome.html>

RDS Multi-AZ deployments -

<https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html>

RDS read replicas - https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_ReadRepl.html

RDS backup and restore -

https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/CHAP_CommonTasks.BackupRestore.html

Amazon Aurora Engineering

Distributed relational storage, reader fleets, Serverless v2, global databases, and resilient application design

VOLUME 6C | CHAPTER 22

Learning Objectives

- Explain Aurora cluster architecture and endpoint behavior.
- Design writer, reader, and custom endpoint usage.
- Evaluate provisioned and Serverless v2 capacity.
- Use global databases, backups, cloning, and failover appropriately.

22.1 Aurora architecture

Amazon Aurora is compatible with selected MySQL and PostgreSQL ecosystems while using an AWS-designed distributed storage system. Compute instances attach to a shared cluster volume, separating much of the storage durability mechanism from individual database instances. This supports rapid replica creation, managed failover, and storage growth within documented limits.

Aurora compatibility does not mean perfect behavioral identity with community engines. Version support, extensions, parameter differences, SQL modes, replication features, and operational tooling must be tested. Treat migration as an application and data project, not as a simple endpoint replacement.

22.2 Cluster endpoints and reader scaling

An Aurora cluster exposes a writer endpoint for the current primary instance and a reader endpoint that distributes connections across available Aurora Replicas. Instance endpoints address a specific member. Custom endpoints can group selected instances for reporting, batch, or tenant isolation.

The reader endpoint balances new connections rather than individual SQL statements. Long-lived pools can remain concentrated on one replica, so application pooling, connection lifetime, and query routing matter. Read-after-write requirements must consider replica lag; critical reads may need the writer or an application consistency mechanism.

22.3 Provisioned and Serverless v2

Provisioned instances provide chosen instance classes and predictable reserved capacity. Aurora Serverless v2 adjusts database compute capacity within a configured range and can be useful for variable, multitenant, development, and unpredictable workloads. It still requires minimum/maximum configuration, monitoring, connection planning, and engine-specific testing.

Serverless scaling is not a substitute for query tuning or schema design. A poor query can consume more capacity and cost while remaining slow. Set capacity ranges from workload evidence, observe scaling and connection behavior, and confirm feature support in the selected engine version and Region.

22.4 Global Database and multi-Region design

Aurora Global Database connects a primary Region with secondary Regions using dedicated cross-Region replication. It can provide local reads and regional recovery capabilities, but application architecture must address write location, DNS or endpoint transition, secrets, networking, promotion procedure, data loss exposure, and business validation.

A global database is not automatically an active-active relational application. Single-writer and conflict behavior depend on the selected features and design. Run controlled switchovers and failovers, measure replication lag, and document which operations are safe during degradation.

22.5 Backups, cloning, and operations

Aurora automated backups and snapshots operate at cluster scope. Fast database cloning can create a copy-on-write cluster for testing, analytics, or release validation without immediately duplicating all underlying data. Clones still incur compute and diverging-storage cost and require independent security controls.

Monitor database load, waits, connections, replica lag, storage, commit latency, deadlocks, and error logs. Use parameter groups under version control, stage upgrades, and rehearse rollback or blue/green patterns where supported. Failover priority tiers should reflect instance suitability and business intent.

Aurora capacity and topology choices

Choice	Best fit	Operational concern
Provisioned writer/readers	Predictable sustained production load	Right-size and reserve capacity
Serverless v2 members	Variable or unpredictable demand	Capacity range, cost, feature support, and scaling behavior
Reader endpoint	General read distribution	Long-lived pools and read-after-write consistency
Custom endpoint	Dedicated reporting or workload class	Membership drift and failover behavior
Global Database	Cross-Region reads and regional recovery	Write topology, lag, promotion, DNS, and data validation

Enterprise Architecture

Global Aurora application

Users -> global traffic routing and regional application stacks

V

Primary Region -> writer endpoint and local readers

V

Secondary Regions -> read endpoints and recovery capacity

Secrets, KMS, network, parameter, and deployment controls in each Region

Replication-lag alarms, switchover runbook, restore tests, and business validation

Aurora inspection commands

```
aws rds describe-db-clusters
aws rds describe-db-instances --filters Name=engine,Values=aurora,aurora-postgresql
aws rds describe-global-clusters
aws rds describe-db-cluster-snapshots
aws rds describe-events --source-type db-cluster --duration 1440
```

Production Best Practices

- Use writer and reader endpoints according to transaction consistency needs.
- Test driver and pool behavior during failover.
- Set Serverless v2 minimum and maximum capacity from evidence.
- Use replicas in multiple Availability Zones.
- Monitor replica lag and reader distribution.
- Protect global database promotion with explicit authority and runbooks.
- Use cloning for safe tests but govern credentials, data sensitivity, and cleanup.
- Rehearse engine upgrades and parameter changes.

Troubleshooting

Symptom	Likely cause	Evidence / action
Reads remain on one replica	Long-lived connection pool	Inspect pool lifetime and use endpoint-aware reconnect behavior
Stale read after write	Replica lag or application routing	Route consistency-critical read to writer or implement session strategy
Serverless cost spike	Wide max range, inefficient SQL, or sustained load	Review capacity metrics, waits, plans, and range
Failover app outage	DNS cache, driver, transaction retry, or secret issue	Trace endpoint resolution, connection retry, and idempotency
Global replica lag grows	Write volume, network, DDL, or long transactions	Inspect global lag metrics and workload changes
Clone contains sensitive data	Environment controls not inherited as expected	Apply independent IAM, network, masking, logging, and expiry

Hands-on Lab: Aurora reader scaling and failover exercise

Objective: Deploy an Aurora cluster with a writer and reader, measure endpoint behavior, test failover, and validate a clone.

Implementation Tasks

1. Create a private encrypted Aurora cluster with writer and reader in separate AZs.
2. Connect using writer, reader, and instance endpoints and record behavior.
3. Generate reads and controlled writes; observe connection distribution and lag.
4. Trigger a supported failover and measure application reconnection.
5. Create a clone for schema-change validation.
6. Optionally test Serverless v2 in a separate low-cost lab configuration.

Validation

- Writer traffic reaches the current writer and read traffic uses approved endpoints.
- Failover changes writer identity and application recovers.
- Clone is isolated and data validation passes.
- Metrics, logs, events, and cleanup evidence are captured.

Cleanup

- Delete clone, reader, cluster, snapshots not retained, secrets, and temporary clients.
- Confirm global or cross-Region resources were not accidentally created.

Interview Questions

1. How does Aurora storage differ conceptually from standard RDS instances?
2. What is the difference among writer, reader, instance, and custom endpoints?
3. Why can a reader endpoint still produce uneven load?
4. When is Serverless v2 appropriate?
5. What does Aurora Global Database provide?
6. Why is a global database not automatically active-active?
7. What is Aurora cloning useful for?
8. How would you test an Aurora failover?

Chapter Summary

- Aurora separates compute members from distributed cluster storage.
- Endpoint selection and connection behavior determine application resilience and consistency.
- Serverless v2 is elastic capacity, not automatic query optimization.
- Global and cloning features require the same security, recovery, and operational discipline as primary databases.

Official AWS References

Amazon Aurora User Guide -

https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/CHAP_AuroraOverview.html

Aurora Serverless v2 - <https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/aurora-serverless-v2.html>

Aurora Global Database - <https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/aurora-global-database.html>

Amazon DynamoDB and NoSQL Design

Access-pattern modeling, partition keys, indexes, capacity, transactions, streams, and global tables

VOLUME 6C | CHAPTER 23

Learning Objectives

- Model DynamoDB tables from access patterns.
- Design partition/sort keys and secondary indexes.
- Choose on-demand or provisioned capacity and diagnose throttling.
- Use transactions, streams, TTL, backups, and global tables safely.

23.1 NoSQL starts with access patterns

DynamoDB is a managed key-value and document database designed for predictable low-latency access at scale. Unlike a relational design that often begins with normalized entities, DynamoDB design begins with the exact requests the application must execute: keys, filters, sort order, cardinality, consistency, latency, and volume.

A table can store several entity types. Composite keys and overloaded indexes can serve multiple access patterns, but they require disciplined naming, documentation, and validation. A scan is not a replacement for a missing access pattern. Filters are applied after items are read and therefore may not reduce consumed capacity as expected.

23.2 Partition and sort keys

The partition key determines the physical distribution of data. High-cardinality, well-distributed keys help avoid hot partitions. The optional sort key groups related items and supports ordered range queries, prefix conditions, and composite relationships. Poor key design can throttle one tenant or timestamp while the rest of the table remains underused.

Write sharding, time bucketing, or tenant-key strategies can distribute extreme workloads, but they increase read and aggregation complexity. The goal is not randomization for its own sake; it is predictable distribution while preserving efficient queries.

23.3 Secondary indexes and consistency

A global secondary index can use different partition and sort keys and is maintained asynchronously. It has its own capacity and storage behavior. A local secondary index shares the table partition key and must be defined at table creation. Index projections should include only what the access pattern needs.

Base-table reads can use eventually consistent or strongly consistent options where supported. Global secondary index reads are eventually consistent. Application design must decide which operations require immediate consistency, conditional writes, version checks, or transaction boundaries.

23.4 Capacity, throttling, and operations

On-demand mode adapts to traffic and simplifies unpredictable workloads. Provisioned mode sets read and write capacity with optional auto scaling and may be cost-effective for stable demand. Adaptive capacity helps uneven access but cannot make an unbounded hot key safe. Monitor consumed capacity, throttled requests, latency, errors, item size, index behavior, and account/table quotas.

DynamoDB transactions provide all-or-nothing operations across multiple items and tables within their documented scope. Conditional expressions support optimistic concurrency and idempotency. Time to Live removes expired items asynchronously and should not be used as a precise scheduler. Streams capture item-level changes for event processing, replication patterns, audit workflows, and materialized views.

23.5 Global tables, backup, and security

Global tables replicate table data across Regions and provide multi-Region, multi-active capabilities according to the selected consistency mode and service features. Conflict handling, transaction boundaries, application idempotency, regional dependency readiness, and cost require explicit design. A multi-Region table does not make every downstream service multi-Region.

Point-in-time recovery and on-demand backups protect against logical and operational failures. Use encryption, private endpoints where appropriate, fine-grained IAM, condition keys, CloudTrail, contributor insights, and data classification. Avoid granting broad table access when applications require only specific actions or leading-key patterns.

DynamoDB design decisions

Decision	Use	Risk to control
Single-table model	Related access patterns and transactional aggregates	Complexity and documentation burden
GSI	Alternative query keys	Asynchronous propagation, cost, and hot index keys
On-demand	Variable or unknown traffic	Cost at sustained high volume
Provisioned + auto scaling	Predictable workload	Forecasting, scale lag, and throttling
Global table	Multi-Region active access	Conflict semantics, cost, and regional dependencies

Enterprise Architecture

Event-driven DynamoDB service

API or service -> conditional writes to DynamoDB

V

Base table and GSIs -> low-latency access patterns

V

DynamoDB Streams -> Lambda/Kinesis consumers and materialized views

V

PITR, backups, exports, and global replicas -> recovery and regional design

V

CloudWatch, Contributor Insights, CloudTrail, alarms, and load tests

DynamoDB inspection and query

```
aws dynamodb list-tables
aws dynamodb describe-table --table-name Orders
aws dynamodb describe-continuous-backups --table-name Orders
aws dynamodb scan --table-name Orders --select COUNT

aws dynamodb query \
  --table-name Orders \
  --key-condition-expression "PK = :pk" \
  --expression-attribute-values '{":pk":{"S":"CUSTOMER#123"}}'
```

Production Best Practices

- Document every access pattern before defining keys.
- Use conditional writes and idempotency for retries.
- Keep item sizes and index projections intentional.
- Load-test hot tenants, time ranges, and burst behavior.
- Prefer queries over scans.
- Monitor each GSI as an independent capacity surface.
- Enable PITR for important tables and test restore.
- Design global-table conflict and regional-failover behavior explicitly.

Troubleshooting

Symptom	Likely cause	Evidence / action
Throttling with low table average	Hot partition or index key	Use Contributor Insights and per-key workload analysis
Query returns no items	Key expression, type, prefix, or index mismatch	Inspect actual attribute types and selected index
GSI result is stale	Asynchronous index propagation	Use base table or workflow that tolerates eventual consistency
Duplicate side effects	At-least-once retry or stream delivery	Use idempotency keys and conditional state transitions
TTL item still exists	TTL deletion is asynchronous	Treat expiration as eligibility, not exact execution time

Symptom	Likely cause	Evidence / action
Global conflict surprise	Concurrent writes in Regions	Define ownership, versioning, or merge semantics

Hands-on Lab: Model and operate a single-table order service

Objective: Implement documented access patterns, a GSI, conditional writes, stream processing, PITR, and a hot-key test.

Implementation Tasks

1. Define customer, order, status, and time-range access patterns.
2. Create a table with composite primary key and one GSI.
3. Load representative items and execute only key-based queries.
4. Use a conditional expression to prevent duplicate order creation.
5. Enable PITR and a stream; process records with a safe idempotent consumer.
6. Run a skewed-load test and inspect throttling or Contributor Insights.
7. Restore the table to a new name and validate data.

Validation

- Every required request maps to a Query/Get/transaction, not a production scan.
- Duplicate writes are rejected safely.
- Stream consumer tolerates retries.
- Restore and hot-key findings are documented.

Cleanup

- Delete restored and source tables, stream consumer, logs, alarms, and test roles after evidence export.
- Retain only approved backups or exports.

Interview Questions

1. Why does DynamoDB design start with access patterns?
2. What causes a hot partition?
3. How do GSIs differ from LSIs?
4. When would you choose on-demand capacity?
5. What consistency options exist?
6. How do conditional writes improve reliability?
7. What are DynamoDB Streams used for?
8. What must be considered with global tables?

Chapter Summary

- DynamoDB performance is primarily a data-model and key-distribution problem.
- Indexes create new access paths and new cost/throttling surfaces.
- Conditional writes, transactions, streams, and idempotency support reliable workflows.
- Global replication and backup require explicit application and recovery design.

Official AWS References

DynamoDB Developer Guide -

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html>

DynamoDB transactions -

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/transactions.html>

DynamoDB global tables -

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/GlobalTables.html>

Amazon ElastiCache and MemoryDB

Caching patterns, Valkey/Redis-compatible systems, Memcached, durability, failover, and application correctness

VOLUME 6C | CHAPTER 24

Learning Objectives

- Differentiate cache and primary in-memory database use cases.
- Choose cache-aside, write-through, and session patterns.
- Design shards, replicas, endpoints, eviction, and failover.
- Operate ElastiCache and MemoryDB securely and observably.

24.1 Cache versus database

A cache stores data that can usually be reconstructed from an authoritative source. Its purpose is to reduce latency, offload databases, and absorb read bursts. An in-memory database can be the authoritative store for selected use cases and therefore requires durability, recovery, and stronger correctness assumptions. Confusing these roles creates data loss or stale-data incidents.

Amazon ElastiCache provides managed distributed cache environments using supported engines such as Valkey, Redis OSS compatibility, or Memcached depending on current service options. Amazon MemoryDB provides a durable, Multi-AZ, Valkey- and Redis OSS-compatible in-memory database. The selection must begin with whether data is disposable, reconstructable, and transactionally authoritative.

24.2 Caching patterns and invalidation

Cache-aside loads data on a miss and writes it into the cache. It is simple and common, but the first request is slow and invalidation must be handled. Write-through updates cache and database together through an application path, while write-behind can improve throughput but increases data-loss and ordering risk.

Time-to-live limits staleness but does not guarantee freshness. Event-driven invalidation, versioned keys, and namespace rotation can improve correctness. Cache stampedes occur when many clients miss simultaneously; request coalescing, jittered TTLs, stale-while-revalidate, and prewarming can reduce the effect. Negative caching must use short, deliberate TTLs.

24.3 Topology, sharding, and failover

Memcached uses a distributed cache model with client-side behavior and is suitable for simple ephemeral caching. Valkey/Redis-compatible deployments support richer data structures, replication, clustering, and persistence options according to service and configuration. Cluster mode partitions keyspace into shards; replicas improve read capacity and availability.

Endpoint selection matters: primary, reader, configuration, and node endpoints serve different purposes. Application clients must support topology discovery and failover. Multi-AZ automatic failover improves availability but cannot make non-idempotent application behavior safe. Resharding and scaling should be tested under load.

24.4 Eviction, memory, and performance

Memory pressure triggers eviction according to the configured policy where supported. A cache with the wrong policy can evict critical sessions or retain low-value entries. Track used memory, fragmentation, evictions, hit rate, CPU, network, connections, replication lag, command latency, and hot keys.

Large values, unbounded collections, expensive commands, and single-threaded engine characteristics can dominate latency. Design key naming, object size, batching, pipelining, and command complexity deliberately. Cache hit rate alone is not sufficient; a high hit rate can still hide stale or incorrect data.

24.5 Security and durable in-memory design

Deploy clusters in private subnets with security-group access only from approved clients. Enable encryption in transit and at rest where supported, authentication, ACLs, managed secrets, and audit controls. Do not expose cache endpoints publicly or assume network location alone is authorization.

MemoryDB is appropriate when an in-memory API and durable primary storage model meet the application requirements. It still requires backup, failover, capacity, shard, and recovery planning. Evaluate service semantics, engine compatibility, durability model, regional availability, and client support before using it as a system of record.

In-memory service selection

Requirement	Likely service/pattern	Key concern
Disposable query cache	ElastiCache cache-aside	Invalidation, stampede, eviction
Simple distributed ephemeral cache	Memcached option	Client distribution and no durable authority
Rich data structures and cache replication	ElastiCache Valkey/Redis-compatible	Cluster-aware client and failover
Durable in-memory primary database	MemoryDB	Cost, durability semantics, backup, and recovery
Session store	Replicated cache or durable design by business need	Session loss tolerance and multi-Region strategy

Enterprise Architecture

Safe cache tier

Application clients with timeouts, retries, and circuit breakers

Private ElastiCache or MemoryDB cluster across AZs

V

Authoritative database or durable MemoryDB model

V

Event-driven invalidation, jittered TTL, prewarm, and idempotency

V

Metrics for hit rate, latency, memory, evictions, lag, errors, and failover

ElastiCache and MemoryDB inspection

```
aws elasticache describe-cache-clusters --show-cache-node-info
aws elasticache describe-replication-groups
aws elasticache describe-events --duration 1440
aws memorydb describe-clusters
aws memorydb describe-snapshots
```

Production Best Practices

- State whether cached data is authoritative or reconstructable.
- Use bounded TTLs with jitter and explicit invalidation.
- Choose cluster-aware clients and test failover.
- Set client timeouts, retry budgets, and circuit breakers.
- Monitor evictions, memory fragmentation, hot keys, and command latency.
- Encrypt traffic and authenticate clients.
- Do not store secrets or irreplaceable data in an unprotected cache.
- Load-test resharding and cache-cold recovery.

Troubleshooting

Symptom	Likely cause	Evidence / action
Database load spikes	Cache cold, mass expiry, or invalidation bug	Inspect hit rate, expirations, deployments, and key namespace
Latency spikes	Hot key, expensive command, CPU, network, or large values	Use engine metrics and slow-command evidence
Unexpected logout	Session key evicted or expired	Inspect eviction policy, memory, TTL, and session design
Failover causes errors	Client not topology-aware or retry unsafe	Validate endpoint type, client discovery, backoff, and idempotency
Memory grows continuously	Unbounded keys or no TTL	Inspect key cardinality, object size, TTL coverage, and fragmentation

Symptom	Likely cause	Evidence / action
Stale data persists	Invalidation failure or excessive TTL	Trace write event, key version, and cache policy

Hands-on Lab: Build a resilient cache-aside layer

Objective: Deploy a private replicated cache, implement safe cache-aside logic, test stampede controls, and observe failover.

Implementation Tasks

1. Create a private encrypted replication group or supported low-cost lab cluster.
2. Configure authentication and a client security group.
3. Implement cache-aside for a small database-backed API.
4. Add jittered TTL, request coalescing, timeout, and fallback.
5. Generate cache-cold and hot-key load; record hit rate and latency.
6. Trigger a supported failover and validate client recovery.
7. Test invalidation after a database update.

Validation

- Cache is not publicly reachable.
- Application remains correct when cache is unavailable.
- Mass expiry does not overwhelm the database beyond the recorded threshold.
- Failover and stale-data behavior are documented.

Cleanup

- Delete cache/MemoryDB resources, snapshots not retained, test database, client host, secrets, and alarms.
- Confirm no reserved capacity or subnet groups remain.

Interview Questions

1. What is the difference between a cache and an in-memory database?
2. How does cache-aside work?
3. What causes a cache stampede?
4. Why add jitter to TTLs?
5. How do shards and replicas differ?
6. What should an application do when the cache is unavailable?
7. When would MemoryDB be considered?
8. Why is hit rate not the only cache metric?

Chapter Summary

- Caching improves latency only when correctness and failure behavior are explicit.
- Topology-aware clients, invalidation, TTL, and stampede controls are application responsibilities.
- ElastiCache is typically a cache tier; MemoryDB can serve durable in-memory database use cases.
- Security, failover, and cache-cold recovery must be tested.

Official AWS References

Amazon ElastiCache User Guide - <https://docs.aws.amazon.com/AmazonElastiCache/latest/dg/WhatIs.html>

Amazon MemoryDB Developer Guide - <https://docs.aws.amazon.com/memorydb/latest/devguide/what-is-memorydb-for-redis.html>

Amazon Redshift and Data Warehousing

Columnar analytics, provisioned and Serverless warehouses, workload management, Spectrum, and data sharing

VOLUME 6C | CHAPTER 25

Learning Objectives

- Explain analytical warehouse architecture and columnar processing.
- Choose provisioned RA3 or Redshift Serverless patterns.
- Design distribution, sort, workload, and ingestion strategies.
- Use Spectrum, data sharing, security, and observability.

25.1 OLTP versus analytical systems

Transactional databases optimize many small concurrent reads and writes with strict transaction behavior. Analytical warehouses optimize scans, joins, aggregations, and large datasets. Moving dashboard and reporting workloads directly onto an OLTP primary can degrade customer transactions. A data warehouse separates analytical compute, storage, governance, and workload priorities.

Amazon Redshift is a managed data warehouse that uses columnar storage and massively parallel processing. Provisioned clusters and Redshift Serverless provide different capacity operating models. Selection depends on usage predictability, concurrency, governance, performance, and cost.

25.2 Data layout and query performance

Columnar storage reduces unnecessary I/O for queries that read selected columns. Compression, data type, sort order, distribution strategy, statistics, table maintenance, and query shape influence performance. Modern Redshift automation can manage many choices, but engineers still need to understand data skew, network redistribution, large joins, and spill behavior.

Distribution determines how rows are placed across compute slices. Poor distribution can concentrate data or force large shuffles. Sort keys influence range elimination and merge behavior. Evaluate query plans and system views rather than applying generic keys to every table.

25.3 Provisioned, Serverless, and workload management

RA3 provisioned nodes separate managed storage from compute and support independent scaling characteristics. Redshift Serverless provisions and scales warehouse capacity for variable analytical workloads. Both require workload limits, cost controls, query governance, and data modeling.

Workload management separates query classes, priorities, and resource behavior. Short dashboard queries, long transformations, ad hoc exploration, and maintenance should not compete without policy. Concurrency scaling and serverless capacity can help, but uncontrolled queries can still create cost and latency incidents.

25.4 Ingestion, Spectrum, and lake integration

COPY from S3 is a high-throughput bulk-ingestion pattern. Streaming ingestion, zero-ETL integrations, DMS, Glue, and partner tools can reduce pipeline latency where supported. Design for idempotency, late data, schema evolution, quarantine, and replay.

Redshift Spectrum queries external data in S3 through catalog metadata. This enables lakehouse patterns but does not eliminate file-layout engineering. Partitioning, columnar formats, compression, file size, catalog quality, and Lake Formation permissions determine performance and governance.

25.5 Data sharing and security

Redshift data sharing can provide live data access across supported clusters, Serverless workgroups, accounts, and Regions without copying the underlying dataset for each consumer. Producer/consumer ownership, write capabilities where supported, object permissions, cost, and version compatibility must be governed.

Use private network paths, encryption, IAM roles, database users/roles, row/column controls, audit logging, and secrets. Separate platform administrators, data owners, analysts, and pipeline identities. Sensitive data should be masked or governed at the correct layer, not merely hidden by dashboard logic.

Warehouse operating models

Model	Best fit	Main control
Provisioned RA3	Predictable sustained enterprise analytics	Node sizing, reservations, workload management
Redshift Serverless	Variable, intermittent, or team-based analytics	RPU/cost limits and query governance
Spectrum external tables	Large lake datasets and infrequent access	File layout, catalog, partitioning, permissions
Data sharing	Live cross-warehouse/account access	Producer/consumer governance and supported topology
Materialized views	Repeated expensive aggregations	Refresh strategy and staleness

Enterprise Architecture

Governed analytics platform

Sources -> batch/stream/zero-ETL ingestion and quality gates

▼

S3 landing/curated zones and Glue Data Catalog

▼

Redshift provisioned or Serverless warehouse

▼

BI tools, notebooks, IAM/database roles, audit logs, cost and query controls**Redshift inspection commands**

```
aws redshift describe-clusters
aws redshift-serverless list-workgroups
aws redshift-serverless list-namespaces
aws redshift describe-cluster-snapshots
aws redshift-data list-statements

# Run read-only SQL through the Data API only with approved credentials/role.
```

Production Best Practices

- Separate transactional and analytical workloads.
- Use columnar formats and appropriately sized files in S3.
- Inspect query plans, skew, redistribution, spills, and queue time.
- Create workload classes and cost limits.
- Use IAM roles instead of embedded access keys.
- Govern data shares and external schemas with explicit ownership.
- Automate data-quality checks and idempotent ingestion.
- Back up metadata, code, permissions, and recovery procedures, not only warehouse data.

Troubleshooting

Symptom	Likely cause	Evidence / action
Query slow after data growth	Skew, poor sort, stale statistics, spill, or bad join	Inspect plan and system views; compare scan, redistribution, and queue time
Serverless cost spike	Unbounded ad hoc queries or high base/max capacity	Review usage limits, query history, and workload controls
COPY errors	Schema mismatch, malformed rows, permissions, encryption, or file format	Inspect load error tables/logs and S3/KMS role access
Spectrum scans too much	Weak partitioning, tiny files, noncolumnar data	Inspect bytes scanned, partitions, file format, and predicates
BI users blocked	WLM queue or connection limit	Inspect queue, concurrency, session behavior, and dashboard polling
Share not visible	Namespace/account/Region permission mismatch	Inspect producer authorization, consumer association, and object grants

Hands-on Lab: Build a governed analytical warehouse

Objective: Load curated S3 data into Redshift, tune a query, create an external table, and enforce workload and access controls.

Implementation Tasks

1. Create a Serverless workgroup or low-cost provisioned lab with private access.
2. Create an IAM role and encrypted S3 landing/curated prefixes.
3. Load a sample dataset with COPY and capture load errors.
4. Run an analytical query, inspect its plan, and improve table design or statistics.
5. Create an external schema/table for columnar S3 data.
6. Apply database roles for loader, analyst, and administrator.
7. Set a usage limit or budget and export audit evidence.

Validation

- No static AWS keys are used.
- Load is idempotent and malformed records are quarantined.
- Tuned query shows measurable improvement.
- Analyst cannot perform administrative actions.
- External query bytes and cost are documented.

Cleanup

- Delete workgroup/cluster, namespace, snapshots not retained, S3 test data, roles, secrets, and log groups.
- Confirm no data share or reserved capacity remains.

Interview Questions

1. Why use a data warehouse instead of the OLTP primary?
2. How does columnar storage help analytics?
3. What are distribution and sort keys?
4. When would you choose Redshift Serverless?
5. What is Redshift Spectrum?
6. What problem does workload management solve?
7. How does data sharing differ from copying data?
8. How would you troubleshoot a suddenly slow Redshift query?

Chapter Summary

- Redshift separates analytical processing from transactional systems.
- Data layout, workload policy, and query governance still matter in a managed warehouse.
- Spectrum and data sharing extend the platform across lakes and organizational boundaries.

- Security and cost controls must be designed with the analytical workflow.

Official AWS References

Amazon Redshift Database Developer Guide -

<https://docs.aws.amazon.com/redshift/latest/dg/welcome.html>

Redshift Serverless - <https://docs.aws.amazon.com/redshift/latest/mgmt/serverless-what-is.html>

Redshift data sharing - <https://docs.aws.amazon.com/redshift/latest/dg/datashare-overview.html>

AWS Data Lakes and Analytics

S3 lake zones, Glue Data Catalog, Athena, Lake Formation, EMR, open table formats, and governance

VOLUME 6C | CHAPTER 26

Learning Objectives

- Design a layered data lake with durable ownership and quality gates.
- Use Glue Data Catalog, crawlers, ETL, and schema governance.
- Query with Athena and process with EMR.
- Apply Lake Formation fine-grained permissions and open table formats.

26.1 Data lake architecture

A data lake stores large volumes of structured, semi-structured, and unstructured data, commonly in Amazon S3. A usable lake is not a bucket filled with files; it is a governed platform with zones, metadata, ownership, quality, lineage, retention, access policy, and cost controls.

Typical zones include landing/raw, standardized, curated, and consumption. Raw data should remain immutable where practical so pipelines can be replayed. Curated data uses stable schemas, partitioning, columnar formats, and business definitions. Separate accounts, buckets, or prefixes according to security boundaries and operational ownership.

26.2 Glue Data Catalog and ETL

The AWS Glue Data Catalog stores table and partition metadata used by Athena, EMR, Redshift Spectrum, and other integrated services. Crawlers can infer schemas, but automatic inference is not governance. Unexpected files can create incorrect types or partitions; production systems should control classifiers, schema evolution, and catalog change review.

AWS Glue ETL can perform serverless data transformation with supported engines and job types. Reliable pipelines are idempotent, checkpointed, observable, and able to quarantine bad records. Data contracts should define field meaning, type, nullability, ownership, freshness, and compatibility.

26.3 Athena and file engineering

Amazon Athena runs SQL queries against data in S3 using catalog metadata. Cost and performance are strongly affected by bytes scanned. Columnar formats such as Parquet or ORC, compression, partition pruning, projection, compact file sizes, and selective queries reduce scanning.

Too many tiny files increase planning and request overhead. Oversized files can reduce parallelism. Partitioning on a high-cardinality or frequently changing field can create catalog and operational problems. Choose partitions from common predicates and lifecycle boundaries, then verify with query statistics.

26.4 EMR and processing choices

Amazon EMR provides managed big-data frameworks such as Apache Spark and related ecosystem tools through several deployment options. Use it for large transformations, machine learning preparation, interactive notebooks, and workloads requiring framework-level control.

Engine choice should follow workload scale, latency, library, governance, and operational skill. Athena is excellent for serverless SQL; Glue is strong for managed ETL; EMR offers broader framework control. A mature platform can use all three against shared cataloged data while maintaining one security and data-quality model.

26.5 Lake Formation and table formats

AWS Lake Formation centralizes data-lake permissions and integrates with services such as Athena, Glue, EMR, and Redshift Spectrum. Fine-grained controls can apply at database, table, row, column, or cell levels in supported configurations. IAM still governs service API access; Lake Formation governs data permissions.

Open table formats such as Apache Iceberg, Hudi, and Delta Lake add transaction, schema-evolution, and time-travel capabilities in supported engines. They also add metadata maintenance, compaction, snapshot retention, and interoperability responsibilities. Choose a format because the workload needs its semantics, not because it is fashionable.

Analytics service selection

Service	Best fit	Primary optimization
Athena	Serverless SQL over S3	Bytes scanned, partitions, file format, workgroups
Glue ETL	Managed batch/stream transformations and catalog	Job design, bookmarks/checkpoints, schema quality
EMR	Spark/Hadoop ecosystem and framework control	Cluster/serverless sizing, parallelism, dependency management
Lake Formation	Centralized lake authorization	Ownership, grants, LF-tags, cross-account governance
Redshift Spectrum	Warehouse users querying lake data	External schema, partitioning, workload integration

Enterprise Architecture

Governed lakehouse

Sources -> immutable S3 landing zone

V

Glue jobs/EMR -> standardized and curated columnar tables

V

Glue Data Catalog + Lake Formation -> metadata and fine-grained authorization

V

Athena, Redshift Spectrum, EMR, BI, and ML consumers

V

Quality, lineage, audit, query limits, lifecycle, compaction, and cost dashboards

Athena and Glue inspection

```
aws glue get-databases
aws glue get-tables --database-name analytics
aws athena list-work-groups
aws athena list-query-executions --work-group primary
aws lakeformation list-permissions
aws emr list-clusters --active
```

Production Best Practices

- Keep raw data immutable and replayable.
- Use columnar compressed formats for analytical zones.
- Choose partitions from actual query predicates and lifecycle needs.
- Control crawler and schema evolution changes.
- Use Athena workgroups with output, encryption, and scan limits.
- Separate IAM API permissions from Lake Formation data permissions.
- Compact small files and maintain table-format metadata.
- Track data owner, freshness, quality, lineage, retention, and cost.

Troubleshooting

Symptom	Likely cause	Evidence / action
Athena scans excessive data	Missing partition pruning or row format	Inspect query predicates, partitions, file format, and bytes scanned
Crawler changes type	Mixed files or inference drift	Use explicit schema/classifier and isolate bad data
Access denied despite IAM allow	Lake Formation grant missing	Inspect LF permissions, resource link, table, and data location
Spark job slow	Skew, small files, shuffle, insufficient parallelism	Inspect stages, partition sizes, spills, and executor metrics
Duplicate output	Retry without idempotent destination	Use deterministic partition/version and commit strategy
Iceberg table grows metadata	No compaction or snapshot expiry	Run supported maintenance and define retention policy

Hands-on Lab: Create a governed S3 analytics lake

Objective: Build landing and curated zones, catalog data, query with Athena, enforce Lake Formation access, and measure scan optimization.

Implementation Tasks

1. Create encrypted landing and curated S3 prefixes with lifecycle and access logging as appropriate.
2. Upload sample CSV data and define a Glue database/table.
3. Transform data to partitioned Parquet using Glue or a local lab tool.
4. Query raw and curated data in separate Athena workgroups and compare bytes scanned.
5. Register the location with Lake Formation and grant analyst read access only to selected columns or tables.
6. Introduce a malformed record and route it to quarantine.
7. Document owner, schema, freshness, quality, and cleanup.

Validation

- Curated query scans materially less data than raw query.
- Analyst access is limited by the intended Lake Formation policy.
- Bad data does not silently corrupt curated output.
- Catalog and S3 permissions are independently understood.

Cleanup

- Delete test tables, workgroups, jobs, EMR resources, S3 objects, keys, and roles after evidence export.
- Retain no query-result bucket or hidden table metadata unintentionally.

Interview Questions

1. What makes a data lake different from a bucket of files?
2. Why keep a raw immutable zone?
3. What is the Glue Data Catalog?
4. How does Athena pricing influence file design?
5. What causes the small-file problem?
6. How do IAM and Lake Formation permissions differ?
7. When would you choose EMR over Athena?
8. What operational work comes with open table formats?

Chapter Summary

- A data lake is a metadata, governance, quality, and ownership system built on durable storage.
- File format, size, partitioning, and catalog design determine analytical efficiency.
- Lake Formation centralizes data authorization across integrated services.
- Open table formats add useful semantics and maintenance obligations.

Official AWS References

AWS Lake Formation - <https://docs.aws.amazon.com/lake-formation/latest/dg/what-is-lake-formation.html>

AWS Glue Data Catalog - <https://docs.aws.amazon.com/glue/latest/dg/catalog-and-crawler.html>

Amazon Athena User Guide - <https://docs.aws.amazon.com/athena/latest/ug/what-is.html>

Amazon EMR Management Guide - <https://docs.aws.amazon.com/emr/latest/ManagementGuide/emr-what-is-emr.html>

Streaming, Kafka, and Search Analytics

Kinesis, Amazon MSK, Data Firehose, OpenSearch, delivery semantics, replay, and real-time operations

VOLUME 6C | CHAPTER 27

Learning Objectives

- Select Kinesis, MSK, Firehose, and OpenSearch patterns.
- Design partitioning, ordering, consumer scaling, and replay.
- Handle at-least-once delivery and schema evolution.
- Operate real-time pipelines with lag, error, and cost controls.

27.1 Streaming system fundamentals

A stream is an ordered sequence of records processed continuously rather than as periodic bulk files. Real-time does not mean instantaneous or exactly once. Production design must define acceptable end-to-end latency, ordering scope, duplicate behavior, replay window, backpressure, schema compatibility, and failure recovery.

Most managed streaming paths deliver at least once in some failure scenarios. Consumers must therefore be idempotent or deduplicate using stable event identifiers and state. Event time, processing time, late arrivals, and out-of-order records should be explicitly handled.

27.2 Kinesis Data Streams and Firehose

Amazon Kinesis Data Streams provides managed streaming with partitions/shards or supported capacity modes, retention, multiple consumers, and replay. Partition keys determine ordering and distribution. A hot partition key can limit throughput even when aggregate capacity appears available.

Amazon Data Firehose delivers streaming data to supported destinations with buffering, transformation, compression, and error delivery. It is appropriate when the goal is managed delivery rather than custom per-record consumer logic. Buffer settings trade latency for efficiency and destination file quality.

27.3 Amazon MSK and Kafka semantics

Amazon Managed Streaming for Apache Kafka operates Kafka clusters or serverless options according to service capabilities. It is useful when applications require Kafka APIs, ecosystem connectors, topic/partition semantics, consumer groups, or portability.

Kafka operations still require topic design, partition count, replication, broker/storage capacity, client compatibility, schema management, ACLs, observability, and upgrade planning. Increasing partitions can change ordering assumptions. Consumer lag measures backlog but must be interpreted with traffic rate and business deadlines.

27.4 OpenSearch ingestion and search design

Amazon OpenSearch Service supports search, log analytics, observability, and vector/search use cases. Documents are indexed into shards and replicas. Index mappings, shard count, rollover, retention, storage tiers, and query patterns determine performance. Too many small shards create overhead; overly large shards slow relocation and recovery.

Data can arrive from Kinesis, Firehose, MSK, OpenSearch Ingestion, agents, or application pipelines. Use dead-letter/error destinations, mapping validation, index templates, and replayable source data. A search index should rarely be the only copy of irreplaceable source records.

27.5 Reliability, schema, and monitoring

Use a schema registry or contract process to manage backward and forward compatibility. Producers should not silently repurpose fields. Consumers should tolerate additive changes and reject or quarantine incompatible data.

Monitor incoming/outgoing throughput, throttling, iterator age or consumer lag, delivery failures, retry volume, dead letters, broker health, shard/index health, JVM/memory, disk, and end-to-end event freshness. Capacity tests must include burst, consumer outage, replay, and downstream degradation.

Real-time service selection

Service	Best fit	Key operational concern
Kinesis Data Streams	AWS-native custom stream processing and replay	Partition keys, capacity, retention, consumer lag
Data Firehose	Managed buffered delivery to destinations	Buffer latency, transformation errors, file quality
Amazon MSK	Kafka APIs and ecosystem	Partitions, brokers, storage, clients, upgrades
OpenSearch Service	Search/log analytics and indexed retrieval	Mappings, shards, rollover, cluster health
S3 durable landing	Replay, audit, batch reprocessing	Delivery latency, partitioning, lifecycle

Enterprise Architecture

Replayable real-time pipeline

Producers with schema and stable event IDs

▼

Kinesis or MSK durable stream

▼

Consumers/OpenSearch Ingestion/Firehose with retries and backpressure

▼

Lag, freshness, DLQ, shard/broker/index health, and end-to-end SLO alarms

Streaming and search inspection

```
aws kinesis list-streams
aws kinesis describe-stream-summary --stream-name events
aws firehose list-delivery-streams
aws kafka list-clusters-v2
aws opensearch list-domain-names
aws opensearch describe-domain --domain-name logs
```

Production Best Practices

- Use stable event IDs and idempotent consumers.
- Choose partition keys from ordering and distribution requirements.
- Keep a replayable durable copy for critical pipelines.
- Define schema compatibility and quarantine invalid events.
- Set retention from recovery and replay objectives.
- Monitor business freshness, not only service health.
- Load-test consumer outage and backlog recovery.
- Design OpenSearch index lifecycle and shard sizing before scale.

Troubleshooting

Symptom	Likely cause	Evidence / action
Iterator age/lag increases	Consumer slow, throttled, failed, or downstream blocked	Compare input rate, consumer errors, capacity, and destination latency
One partition hot	Skewed partition key	Inspect key distribution and redesign/shard carefully
Duplicate records	At-least-once retry	Deduplicate by event ID and idempotent state transition
Firehose error objects grow	Transform, permission, schema, or destination failure	Inspect error prefix, Lambda logs, destination and KMS access
Kafka rebalance storm	Consumer instability or session/config mismatch	Inspect consumer logs, lag, group events, and network
OpenSearch cluster red/yellow	Unassigned shards, disk, node loss, or mapping explosion	Inspect cluster allocation, storage, shard count, and index mappings

Hands-on Lab: Build a replayable event pipeline

Objective: Publish partitioned events, process them idempotently, archive to S3, index selected fields, and recover from consumer failure.

Implementation Tasks

1. Create a Kinesis stream or low-cost supported MSK lab.
2. Define an event schema and stable event ID.
3. Publish distributed and intentionally skewed traffic.
4. Create a consumer that writes idempotently to a sink.
5. Archive raw events to S3 through Firehose or a controlled consumer.
6. Index selected events into OpenSearch or simulate the index step if cost-constrained.
7. Stop the consumer, build backlog, restart, and measure catch-up.

Validation

- Ordering is preserved only within the documented key/partition scope.
- Duplicate delivery does not duplicate business state.
- Archived records support replay.
- Lag and recovery time are measured.

Cleanup

- Delete streams, delivery streams, MSK/OpenSearch resources, buckets, roles, logs, and endpoints.
- Check for provisioned capacity, ENIs, and retained snapshots.

Interview Questions

1. What does at-least-once delivery mean for consumers?
2. How does a partition key affect Kinesis?
3. When would you choose MSK instead of Kinesis?
4. What is consumer lag?
5. Why archive streaming data to S3?
6. What is the purpose of a schema registry?
7. How do OpenSearch shards affect operations?
8. How would you recover from a six-hour consumer outage?

Chapter Summary

- Streaming design is defined by ordering, duplication, latency, replay, and backpressure.
- Kinesis and MSK provide different APIs and operating models.
- Firehose simplifies managed delivery; OpenSearch provides indexed search and analytics.
- Idempotency, schema governance, and replay tests are mandatory for reliable real-time systems.

Official AWS References

Kinesis Data Streams Developer Guide - <https://docs.aws.amazon.com/streams/latest/dev/introduction.html>

Amazon MSK Developer Guide - <https://docs.aws.amazon.com/msk/latest/developerguide/what-is-msk.html>

Amazon OpenSearch Service Developer Guide -

<https://docs.aws.amazon.com/opensearch-service/latest/developerguide/what-is.html>

AWS Database Migration Engineering

Discovery, assessment, schema conversion, full load, change data capture, cutover, and rollback

VOLUME 6C | CHAPTER 28

Learning Objectives

- Plan homogeneous and heterogeneous database migrations.
- Use DMS discovery, schema conversion, full load, and CDC.
- Design validation, cutover, rollback, and performance controls.
- Avoid common migration security and consistency failures.

28.1 Migration is a program, not a copy job

Database migration changes a live dependency with data, schema, code, operational, security, performance, and organizational consequences. Discovery must identify engines, versions, size, growth, extensions, stored code, jobs, dependencies, peak rates, network paths, compliance, RPO/RTO, maintenance windows, and business owners.

A homogeneous migration keeps the same or compatible engine and is often simpler. A heterogeneous migration changes engine and requires schema and code conversion. Replatforming may alter features, query plans, transaction behavior, collation, identity generation, time zones, data types, and operational tooling.

28.2 DMS components and migration modes

AWS Database Migration Service can discover, assess, convert supported schemas, and migrate data. A migration uses source and target endpoints, replication capacity or supported serverless options, and one or more tasks. Full load copies existing rows; change data capture reads ongoing changes to keep the target synchronized.

Full load plus CDC is a common low-downtime pattern. It still requires source log configuration, retention, primary keys or supplemental logging, target preparation, network throughput, and task tuning. DMS is not a general application transformation engine; complex business transformation may belong in ETL or application migration code.

28.3 Schema conversion and assessment

DMS Schema Conversion can assess and convert supported source schemas to compatible target formats. AWS SCT remains available for supported scenarios, but current AWS guidance should be checked when selecting tooling. Assessment reports identify automatically converted and manual-action objects.

Stored procedures, functions, triggers, proprietary types, optimizer hints, packages, and application SQL can dominate effort. Teams should classify each object: convert automatically, rewrite, retire, replace with service capability, or retain temporarily. The assessment report is the beginning of engineering work, not proof of compatibility.

28.4 Validation and cutover

Validation must cover row counts, checksums or sampled hashes, key business aggregates, referential integrity, LOBs, sequences, permissions, code objects, time zones, encoding, and application transactions. DMS validation features can help, but business-level reconciliation remains essential.

Cutover normally includes change freeze, final deployment, CDC lag drain, source write stop, final validation, endpoint/DNS/secret switch, smoke tests, enhanced monitoring, and a rollback decision point. Rollback becomes harder once writes occur on the target, so the data ownership rule after cutover must be explicit.

28.5 Security, performance, and failure handling

Use private network paths, TLS, secrets, least-privilege roles, encrypted replication storage, and controlled logs. Migration accounts and roles should be temporary and removed after acceptance. Avoid opening broad database access for convenience.

Monitor task state, table progress, throughput, latency, CDC source/target lag, errors, memory, storage, network, and source impact. Large objects, absent keys, long transactions, DDL, unsupported data types, and target constraints commonly cause failure. Rehearse the migration with production-like data and rate before the final event.

Migration strategy selection

Pattern	Downtime profile	Main work
Backup/restore	Higher or planned outage	Backup transfer, restore, upgrade, validation
Native replication	Low downtime for compatible engines	Replication config, lag, promotion
DMS full load	Planned cutover after copy	Task tuning and validation
DMS full load + CDC	Low downtime	Log retention, lag drain, cutover control
Heterogeneous DMS + schema conversion	Low/moderate downtime	Schema/code remediation and extensive testing

Enterprise Architecture

Low-downtime database migration

Discovery and assessment -> dependency inventory and migration wave

V

Schema conversion -> report, remediation backlog, target build

V

Private DMS replication -> full load and CDC

V

V

Cutover authority -> freeze, lag zero, switch, smoke test, observe, rollback/fail-forward

DMS inspection commands

```
aws dms describe-replication-instances
aws dms describe-replications
aws dms describe-replication-tasks
aws dms describe-endpoints
aws dms describe-table-statistics --replication-task-arn TASK_ARN
aws dms describe-events --duration 1440
```

Production Best Practices

- Inventory application SQL and stored code, not only tables.
- Run an assessment and maintain a manual-remediation backlog.
- Use private encrypted connectivity and temporary least privilege.
- Rehearse with production-like data volume and change rate.
- Track CDC lag and source log retention.
- Validate business aggregates and transactions.
- Define a point after which rollback is no longer safe.
- Remove migration roles, endpoints, replication capacity, and temporary access after acceptance.

Troubleshooting

Symptom	Likely cause	Evidence / action
Task stops	Endpoint, permission, unsupported object, or resource exhaustion	Inspect task logs, event, endpoint test, memory/storage, and table state
CDC lag grows	Source rate, target apply, network, large transaction	Compare source/target latency and identify long transactions
Rows differ	Filters, errors, LOB mode, missing keys, or concurrent change	Use table statistics, validation logs, and business reconciliation
Target slow after cutover	Indexes/statistics/config/query plan differ	Capture plans, statistics, parameter and workload comparison
Schema object missing	Conversion unsupported or manual action ignored	Review assessment report and remediation tracker
Rollback unsafe	Writes occurred on target without reverse sync	Follow declared ownership and fail-forward/dual-write plan

Hands-on Lab: Migrate a relational database with full load and CDC

Objective: Assess a source, prepare a target, perform full load plus CDC, reconcile data, and execute a rehearsed cutover.

Implementation Tasks

1. Create small source and target databases in private subnets.
2. Inventory schema objects, users, and application queries.
3. Run DMS Schema Conversion assessment if engines differ.
4. Create DMS endpoints and replication capacity with TLS and least privilege.
5. Start full load plus CDC and generate ongoing source writes.
6. Track table statistics and CDC lag; fix one induced data issue.
7. Execute cutover runbook and validate business transactions.

Validation

- Source and target row counts and key aggregates reconcile.
- CDC lag reaches the approved threshold before switch.
- Application connects with new secret/endpoint and passes smoke tests.
- Rollback decision and data ownership are documented.

Cleanup

- Delete tasks, endpoints, replication capacity, temporary roles, logs, source/target lab databases, and secrets.
- Retain assessment and reconciliation evidence only.

Interview Questions

1. What is the difference between homogeneous and heterogeneous migration?
2. How do full load and CDC work together?
3. What does schema conversion not solve automatically?
4. Why are source transaction logs important?
5. How do you validate a database migration?
6. What is a rollback decision point?
7. Why can the target perform differently with identical data?
8. What migration resources must be removed after completion?

Chapter Summary

- Database migration requires discovery, conversion, replication, validation, and business cutover control.
- DMS can reduce data-movement work but does not remove schema and application remediation.
- CDC lag, source impact, and business reconciliation are critical evidence.

- Rollback and post-cutover data ownership must be decided before the event.

Official AWS References

AWS DMS User Guide - <https://docs.aws.amazon.com/dms/latest/userguide/Welcome.html>

DMS Schema Conversion -

https://docs.aws.amazon.com/dms/latest/userguide/CHAP_SchemaConversion.html

AWS Schema Conversion Tool - https://docs.aws.amazon.com/SchemaConversionTool/latest/userguide/CHAP_Welcome.html

Data Security, Backup, Reliability, and Operations

Identity, encryption, secrets, observability, backup, recovery, cost, and database SRE practices

VOLUME 6C | CHAPTER 29

Learning Objectives

- Create a consistent security model across AWS data services.
- Design backup, replication, and restore testing from RPO/RTO.
- Operate databases with SLI/SLO, capacity, change, and incident controls.
- Govern cost, data retention, and compliance evidence.

29.1 Data security layers

Database security combines identity, authorization, network paths, encryption, secrets, audit, patching, configuration, and data-level controls. No single layer is sufficient. A private endpoint does not authorize a query; encryption does not prevent a privileged user from reading plaintext; IAM permission to call a service API is not always the same as database permission inside the engine.

Separate platform administrators, database administrators, security teams, application identities, analysts, and backup operators. Use federated human access, workload roles, short-lived credentials where supported, managed secrets, and break-glass procedures with monitoring. Remove shared administrator accounts and static keys.

29.2 Encryption and KMS operations

Encrypt data at rest and in transit. KMS key policies, grants, aliases, rotation, deletion safeguards, and cross-account access are part of availability. A backup copied to another account or Region is unusable if the recovery team cannot decrypt it.

Use customer managed keys when control, separation, or cross-account policy requires them; otherwise evaluate AWS owned or managed options according to governance. Test encryption paths during restore and failover, not only during creation. Certificate validation and TLS mode should be enforced by clients.

29.3 Backup, replication, and restore testing

Backup protects against logical deletion, corruption, ransomware, operator error, and some platform failures. Replication improves availability and read locality but can replicate bad data. Snapshots, point-in-time recovery, exports, native dumps, cross-account copies, and AWS Backup serve different needs.

Define RPO and RTO per data product. A restore test must include database creation, keys, network, parameters, users/roles, secrets, DNS/endpoints, application schema, and business validation. Record actual recovery point, duration, manual steps, and data-quality findings.

29.4 Database SRE and observability

Service-level indicators can include availability, successful transaction rate, p95/p99 query latency, replication lag, freshness, backup success, restore duration, and error budget consumption. Infrastructure metrics alone do not describe data correctness.

Capacity planning tracks growth, working set, I/O, connections, queueing, cache efficiency, shard/partition distribution, and maintenance windows. Change management covers schema migration, engine version, parameter, scaling, index, permissions, and data-retention changes. Automate checks but keep accountable approval for high-risk operations.

29.5 Cost, retention, and incident response

Data cost includes compute, storage, backup copies, I/O, requests, data transfer, cross-Region replication, license, query scans, idle clusters, and retained logs. Tag and allocate by product/owner, set budgets and anomaly detection, and tie retention to legal and business requirements.

Incident runbooks should distinguish availability, performance, data corruption, unauthorized access, and migration failures. Preserve evidence, stop harmful automation, identify the authoritative copy, communicate data-loss exposure, and validate recovery before reopening writes. Post-incident actions must address technical and process causes.

Data protection controls

Failure mode	Primary protection	Required test
Instance/AZ failure	Multi-AZ or replica failover	Application reconnection and transaction continuity
Accidental deletion	PITR/version/backup	Restore to isolated environment
Data corruption	Independent backup and validation	Business-level reconciliation
Region outage	Cross-Region replica/copy and recovery stack	Regional game day and DNS/secret transition
Credential compromise	Least privilege, rotation, audit, isolation	Access-revocation and incident exercise
Cost runaway	Budgets, limits, retention, anomaly detection	Alert routing and owner response

Enterprise Architecture

Data operations control plane

Federated humans and workload roles -> least privilege

V

Private endpoints, TLS, KMS, secrets, database authorization

V

Metrics, logs, audit, query performance, data quality, and freshness

V

Backup plans, cross-account vault/copies, restore automation, game days

V

Budgets, retention, ownership, incident command, and compliance reports

Cross-service operations checks

```
aws cloudwatch describe-alarms --state-value ALARM
aws backup list-backup-jobs
aws backup list-restore-jobs
aws kms list-aliases
aws secretsmanager list-secrets
aws cloudtrail lookup-events --max-results 20
aws ce get-cost-and-usage --time-period Start=2026-07-01,End=2026-08-01 --granularity MONTHLY --
metrics UnblendedCost
```

Production Best Practices

- Define data owner, classification, RPO, RTO, retention, and residency.
- Separate human, workload, backup, and break-glass identities.
- Test KMS and secret access during recovery.
- Measure business transactions and data freshness.
- Use schema/version changes through pipelines and migrations.
- Restore into isolation and validate before production use.
- Alert on backup selection drift and failed jobs.
- Track full data-platform cost including transfer and retained copies.

Troubleshooting

Symptom	Likely cause	Evidence / action
Backup green but restore unusable	Missing key, role, parameter, schema, or dependency	Perform complete isolated restore and application validation
Metrics normal but users see errors	Business/data-quality issue not covered by infrastructure metrics	Inspect transaction success, freshness, query errors, and application logs
Cross-account copy fails	Vault/key policy or organization control	Trace role, vault access, KMS grants, SCP, and CloudTrail
Unexpected cost	Scan, I/O, transfer, replicas, idle capacity, or retention	Break down by service, usage type, tag, and owner
Unauthorized query	Overbroad DB role or shared credential	Audit database/user activity, revoke, rotate, and review grants

Symptom	Likely cause	Evidence / action
Recovery creates more damage	Wrong authoritative copy or writes reopened early	Stop writes, declare source of truth, reconcile, and stage validation

Hands-on Lab: Run a data-platform recovery and security game day

Objective: Demonstrate least privilege, backup restoration, key access, application validation, and incident evidence across one relational and one NoSQL/analytics service.

Implementation Tasks

1. Select two small lab data services and define owner, classification, RPO, RTO, and retention.
2. Create least-privilege workload and operator roles.
3. Enable encryption, logging, alarms, and backup/PITR.
4. Inject an accidental data deletion or overwrite in test data.
5. Restore to isolated resources and validate data plus application behavior.
6. Simulate a missing KMS permission and resolve through the approved role.
7. Measure actual recovery and produce incident/postmortem notes.

Validation

- Workload cannot perform administrative backup/key actions.
- Restored data meets declared business checks.
- Actual RPO/RTO are recorded.
- CloudTrail/service/database evidence identifies the change and recovery.
- Cost and cleanup inventory are complete.

Cleanup

- Delete restored resources, test data, temporary grants, roles, alarms, and copies according to retention.
- Do not schedule KMS key deletion until every dependent recovery point is reviewed.

Interview Questions

1. Why is a private endpoint not sufficient database security?
2. How can KMS become a recovery dependency?
3. Why is replication not the same as backup?
4. What should a database restore test validate?
5. What are useful database SLOs?
6. How do you monitor data freshness?
7. What causes data-platform cost surprises?
8. How should an incident team choose the authoritative copy?

Chapter Summary

- Data security is layered across identity, network, encryption, secrets, authorization, and audit.
- Recovery success requires keys, configuration, identity, and application validation.
- Database SRE measures transaction and data outcomes, not only infrastructure.
- Ownership, retention, cost, and incident authority are part of production engineering.

Official AWS References

AWS Well-Architected Security Pillar - <https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/welcome.html>

AWS Backup Developer Guide - <https://docs.aws.amazon.com/aws-backup/latest/devguide/whatisbackup.html>

AWS KMS Developer Guide - <https://docs.aws.amazon.com/kms/latest/developerguide/overview.html>

Enterprise Data Platform Architecture and Capstone

Selecting services, integrating transactional and analytical systems, governance, reliability, and end-to-end delivery

VOLUME 6C | CHAPTER 30

Learning Objectives

- Select AWS data services from workload semantics.
- Design an enterprise data platform with transactional, cache, lake, warehouse, and streaming layers.
- Apply governance, security, observability, cost, and recovery across accounts.
- Complete a production-style capstone and architecture review.

30.1 Service selection from semantics

The first architecture question is not which AWS service is popular. It is what the workload requires: relational transactions, key-value access, graph traversal, time series, search, in-memory latency, analytical scans, stream processing, file/object storage, consistency, scale, and recovery. One application can use several data stores, but each additional system creates synchronization, security, cost, and operational complexity.

Use the fewest services that satisfy requirements. Polyglot persistence is valuable when each store has a clear bounded responsibility and data ownership. It becomes harmful when teams duplicate data without lineage, conflict rules, or recovery authority.

30.2 Transactional, event, and analytical boundaries

Transactional services own current business state in RDS, Aurora, DynamoDB, or another fit-for-purpose database. A cache accelerates access but normally does not become the hidden source of truth. Domain events publish durable changes to Kinesis or MSK.

Raw events and extracts land in S3 for replay and analytics. Glue/EMR transform them into curated tables. Athena and Redshift serve analytical consumers. OpenSearch provides indexed retrieval and log/search experiences. Every derived store must be rebuildable or have an independent recovery model.

30.3 Multi-account governance

A mature platform separates production applications, data lake, analytics, security/log archive, shared services, and backup responsibilities across accounts or organizational units where justified. Lake Formation, IAM Identity Center, Organizations policies, KMS, Resource Access Manager, and service resource policies establish governed cross-account access.

Centralization should not create an unowned bottleneck. Platform teams provide paved roads: templates, modules, approved engines, network patterns, encryption, backup, observability, cost tags, and automated policy checks. Data product teams remain accountable for schema, quality, freshness, access approval, and business recovery.

30.4 Reliability and delivery lifecycle

Each data product needs SLOs, capacity model, change pipeline, backup policy, restore runbook, incident ownership, and cost budget. Schema changes are versioned and compatible. Pipelines are idempotent and replayable. Streaming consumers handle duplicates and backpressure. Analytical tables have quality and freshness checks.

Architecture reviews should challenge failure modes: AZ loss, Region loss, bad deployment, deleted table, compromised credential, KMS denial, hot partition, cache outage, consumer lag, malformed schema, runaway query, and incomplete migration. A design is not complete until the team can explain detection, containment, recovery, and evidence.

30.5 Capstone acceptance model

The capstone builds a small commerce data platform: Aurora or RDS for orders, DynamoDB for a high-scale access pattern, ElastiCache for acceleration, Kinesis for events, S3/Glue/Athena for a lake, and optional Redshift/OpenSearch components according to cost limits.

Acceptance is based on measurable behavior, not screenshots: private connectivity, least privilege, encrypted data, successful transactions, idempotent event processing, analytical freshness, backup restore, failover recovery, query/cost limits, and complete cleanup. Infrastructure as Code is strongly recommended so the environment can be rebuilt and reviewed.

AWS data service decision guide

Requirement	Preferred starting point	Challenge question
Relational transactions and SQL	RDS or Aurora	Engine compatibility, HA, connection and migration needs?
Predictable key-value/document access	DynamoDB	Can every access pattern be expressed by keys/indexes?
Microsecond/low-millisecond cache	ElastiCache	Is data reconstructable and invalidation correct?
Durable in-memory primary	MemoryDB	Do durability semantics and cost fit?
Data warehouse	Redshift	Provisioned or Serverless, workload and lake integration?
Serverless SQL on lake	Athena	File layout, partitions, scan limit, governance?
Streaming events	Kinesis or MSK	Ordering, replay, APIs, ecosystem, operating model?

Requirement	Preferred starting point	Challenge question
Search/log analytics	OpenSearch	Index lifecycle, shard sizing, source-of-truth model?

Enterprise Architecture

Enterprise commerce data platform

Clients -> API and application services in private networks

V

Aurora/RDS orders + DynamoDB access model + ElastiCache acceleration

V

Transactional outbox/events -> Kinesis/MSK -> idempotent consumers

V

S3 raw/curated lake -> Glue Catalog/Lake Formation -> Athena/Redshift

V

OpenSearch for selected search/analytics use cases

V

Central identity, KMS, secrets, logs, backups, budgets, SLOs, and recovery game days

Capstone evidence commands

```
aws sts get-caller-identity
aws rds describe-db-clusters
aws dynamodb list-tables
aws elasticache describe-replication-groups
aws kinesis list-streams
aws glue get-databases
aws athena list-work-groups
aws backup list-backup-jobs
aws cloudwatch describe-alarms --state-value ALARM
```

Production Best Practices

- Start with workload semantics and bounded data ownership.
- Keep derived data replayable or independently recoverable.
- Use private connectivity, TLS, managed secrets, and least privilege.
- Build schema and infrastructure changes through versioned pipelines.
- Measure transaction success, latency, freshness, lag, quality, and cost.
- Test cache outage, database failover, consumer backlog, data deletion, and restore.
- Use account boundaries for blast radius and governance where justified.

- Create a complete deletion and retention plan before the capstone begins.

Troubleshooting

Symptom	Likely cause	Evidence / action
Services work individually but workflow fails	Contract, identity, network, or event boundary mismatch	Trace one business transaction end to end with correlation ID
Analytics stale	Pipeline delay, consumer lag, partition/catalog issue	Measure source event time through curated/warehouse availability
Duplicate orders	Retry and event consumer not idempotent	Use transaction/outbox and conditional business key
Cache outage causes full failure	Cache treated as mandatory authority	Implement fallback, timeout, circuit breaker, and load protection
Recovery restores DB but not platform	Streams, catalog, secrets, roles, or derived stores missing	Use full dependency runbook and replay
Costs exceed lab budget	Unbounded warehouse/search/cache or retained resources	Apply budgets, serverless limits, expiry tags, and teardown inventory

Hands-on Lab: Volume 6C capstone - production-style commerce data platform

Objective: Build, test, secure, observe, recover, and document an integrated AWS data platform using at least five services from Part C.

Implementation Tasks

1. Write workload requirements, data ownership, access patterns, SLOs, RPO/RTO, classification, budget, and cleanup date.
2. Deploy a private relational order database and one DynamoDB access pattern.
3. Add a cache-aside layer that remains optional for correctness.
4. Publish order events to Kinesis/MSK with stable IDs and process idempotently.
5. Archive events to S3, catalog curated data, and query with Athena; add Redshift/OpenSearch only if within budget.
6. Implement least privilege, managed secrets, TLS, encryption, audit logs, alarms, backup/PITR, and cost limits.
7. Inject database failover, cache outage, duplicate event, consumer backlog, malformed record, and data deletion.
8. Restore and replay; measure transaction continuity, freshness, lag, RPO, RTO, and cost.
9. Produce architecture, threat model, runbooks, evidence, postmortem, and cleanup report.

Validation

- No service is publicly exposed without documented need.
- No long-term AWS key or plaintext database secret is used.
- Duplicate events do not duplicate business state.
- Cache failure degrades performance but not correctness.
- Analytics freshness and bytes/cost are measured.
- Database recovery and event replay meet stated objectives.
- Every alarm, failure, and recovery has evidence and accountable action.
- Teardown leaves no chargeable orphan.

Cleanup

- Export code, diagrams, logs, metrics, validation, and lessons learned.
- Delete consumers, search/warehouse resources, streams, catalogs, buckets, cache, DynamoDB, relational databases, backups according to retention, secrets, endpoints, roles, keys, and logs.
- Review Cost Explorer/resource inventory after teardown.

Interview Questions

1. How do you choose among RDS, Aurora, DynamoDB, and MemoryDB?
2. What is a bounded data owner?
3. Why should a cache not become a hidden source of truth?
4. How do you make an event pipeline replayable?
5. What does a governed data lake require beyond S3?
6. How do you measure analytical freshness?
7. What account boundaries would you use for an enterprise data platform?
8. How would you test end-to-end recovery?

Chapter Summary

- Part C connects transactional, NoSQL, cache, warehouse, lake, streaming, migration, and operations disciplines.
- Service selection begins with semantics, consistency, access patterns, and ownership.
- Reliable platforms make changes versioned, events idempotent, data governed, and recovery measurable.
- The capstone demonstrates an integrated system rather than isolated service configuration.

Official AWS References

AWS Databases decision guidance - <https://aws.amazon.com/products/databases/>

AWS Analytics documentation - <https://docs.aws.amazon.com/analytics/>

AWS Well-Architected Framework -

<https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html>

Appendix A - AWS Data Services CLI Reference

Area	Read-only command examples
RDS/Aurora	aws rds describe-db-instances describe-db-clusters describe-events
DynamoDB	aws dynamodb list-tables describe-table describe-continuous-backups
ElastiCache/MemoryDB	aws elasticache describe-replication-groups aws memorydb describe-clusters
Redshift	aws redshift describe-clusters aws redshift-serverless list-workgroups
Lake	aws glue get-databases aws athena list-work-groups aws lakeformation list-permissions
Streaming	aws kinesis list-streams aws kafka list-clusters-v2 aws firehose list-delivery-streams
OpenSearch	aws opensearch list-domain-names describe-domain
Migration	aws dms describe-replication-tasks describe-endpoints
Operations	aws backup list-backup-jobs aws cloudwatch describe-alarms

Appendix B - Cross-service Troubleshooting Matrix

Layer	Question	Evidence
Identity	Who is the caller and what denies apply?	STS, IAM, resource policy, DB grants, Lake Formation, CloudTrail
Network	Can client resolve and reach the private endpoint?	DNS, SG, routes, NACL, flow logs, TLS test
Capacity	Which resource is saturated or skewed?	CPU, memory, I/O, connections, partitions, shards, queue/lag
Application	Are retries, pools, consistency, and idempotency correct?	Trace, driver/pool config, correlation ID, transaction logs
Data	Is schema, quality, freshness, and ownership correct?	Catalog, constraints, reconciliation, lineage, event timestamps
Recovery	Can the point be decrypted and used by the application?	Restore job, KMS, secrets, DNS, business validation
Cost	Which usage type and owner caused spend?	Cost Explorer, CUR, tags, query scans, retained copies

Appendix C - Production Readiness Checklist

Transactional databases

- Private endpoints and controlled clients.
- Multi-AZ/read scaling selected separately.
- Managed secrets, TLS, encryption, backups, failover and restore tested.
- Schema and engine changes use pipelines.

NoSQL and cache

- Access patterns and partition keys documented.
- Idempotency, TTL, invalidation, failover, and hot-key tests completed.
- PITR/backup and cache-cold behavior verified.

Analytics and lake

- Raw and curated ownership defined.
- Columnar files, partitions, catalog, quality, and Lake Formation grants reviewed.
- Workgroup/query/cost limits and small-file maintenance enabled.

Streaming and search

- Schema and event IDs defined.
- Lag, replay, DLQ, backpressure, and duplicate handling tested.
- Index lifecycle and shard sizing documented.

Migration and recovery

- Assessment, reconciliation, cutover, rollback authority, and cleanup approved.
- RPO/RTO measured through full application recovery.

Appendix D - Glossary

Term	Definition
CDC	Change data capture that replicates ongoing source changes.
Data Catalog	Central metadata repository for databases, tables, and partitions.
GSI	DynamoDB global secondary index with alternative keys.
Idempotency	Repeated processing produces the same final business state.
Lake Formation	AWS service for governed data-lake setup and permissions.
OLTP	Online transaction processing for operational transactions.
PITR	Point-in-time recovery within a retained window.
RPO	Maximum acceptable data loss measured in time.

Term	Definition
RTO	Target time to restore service.
Shard/partition	Unit used to distribute streaming, NoSQL, or search data/work.
WLM	Workload management for analytical query classes.
Zero-ETL	Managed integration that reduces custom extraction/transform pipeline work in supported scenarios.

Appendix E - Official Documentation Families

- Amazon RDS and Aurora - <https://docs.aws.amazon.com/rds/>
- Amazon DynamoDB - <https://docs.aws.amazon.com/dynamodb/>
- Amazon ElastiCache - <https://docs.aws.amazon.com/elasticache/>
- Amazon MemoryDB - <https://docs.aws.amazon.com/memorydb/>
- Amazon Redshift - <https://docs.aws.amazon.com/redshift/>
- AWS Glue, Athena, Lake Formation, and EMR - <https://docs.aws.amazon.com/analytics/>
- Kinesis, MSK, and OpenSearch - <https://docs.aws.amazon.com/>
- AWS Database Migration Service - <https://docs.aws.amazon.com/dms/>
- AWS Backup - <https://docs.aws.amazon.com/aws-backup/>

End of Volume 6C

You have completed AWS Database and Data Platform Engineering. Volume 6D continues with containers, serverless computing, messaging, event orchestration, and application delivery services.